BursaIQ

REPOSITORY-ALIGNED · OCT 2026

Technical Architecture & Implemented Platform

Flask APIs, governed AI execution, deterministic analytics, role-filtered knowledge, human approval and reusable intelligence—implemented as one local prototype.

Python 3.11+ | Flask | SQLite | JavaScript | Ollama | Copilot Studio | RAG | PDF output

Scope: code architecture, feature wiring, data paths, governance controls and demo boundaries

IMPLEMENTED SURFACE

Twelve connected pages share one governed backend

Navigation is grouped by execution, follow-through and administration; all stateful features persist through Flask APIs.

ENTRY

Home

Live semantic routing to conversational Q&A or an active registered mission capability.

EXECUTE

AI Assistant

Live answers, evidence, metrics and hybrid HTML/SVG analytics charts.

KNOWLEDGE

Ask Bursa

Role-filtered RAG over indexed documents with citations and source-scope disclosure.

MISSIONS

My Workforce

Governed missions, worker stages, agents, teams and capabilities.

COLLAB

Collaborative Rooms

Persisted rooms, tasks, files, AI teammates and decisions.

WORKFLOW

Automation Studio

Persisted visual workflows with validation, retries, approvals and resumable runs.

REUSE

Intelligence Exchange

Approved, versioned assets with provenance and traceable reuse as a new mission.

FOLLOW-UP

Action

Backend-derived attention queue, decision memory and reviewable email schedules.

GOVERN

Verification Centre

Review, approval, requested changes, resubmission and audit history.

ADMIN

Knowledge Admin

Upload, metadata, permissions, indexing, failures, removal and suggestions.

ADMIN

Connectors

Governed requests; external access stays unavailable until configured.

PLATFORM

Settings

Local preferences and live model-provider availability.

Shared rendering contract

Summary metrics · tables · charts · findings · evidence · quality checks · interpretation boundaries · downloadable outputs

All surfaces are implemented in index.html + static/js/workforce.js and backed by server.py / bursaiq modules.

DESIGN ARCHITECTURE

Layered system with explicit execution and trust boundaries

The model interprets intent and generates language; Python owns capability allowlisting, deterministic analytics, validation, persistence and reports.

1 **Browser interface** — Responsive HTML/CSS/JavaScript · 11 pages · safe Markdown · HTML/SVG analytics charts

2 **Flask API boundary** — Routing, request validation, role checks, provider gateway and feature-specific APIs

3 **Orchestration & governance** — Mission engine · Agent Registry · teams · Capability Registry · workflow engine · approval gates

4 **Execution services** — Allowlisted Python executors · RAG · PDF generation · model providers · structured result validation

5 **State & governed assets** — SQLite runtime store · controlled knowledge files · synthetic CSV inputs · output/pdf deliverables

LANGUAGE

Model providers

Ollama: qwen2.5:3b + nomic-embed-text
Copilot Studio: server-side Direct Line token exchange

ALLOWLIST

Capability safety boundary

A registry record cannot install code. Only executor IDs in MISSION_EXECUTORS can run; missing, disabled, changed or input-incomplete contracts are rejected.

STATE

Persistence boundary

SQLite persists governed platform state. Local Analysis uploads remain in memory; reports are written to output/pdf/.

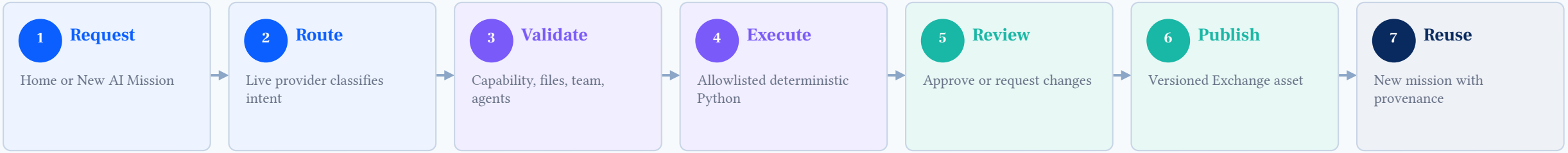
NO SECRETS IN BROWSER

NO SILENT FAKE SUCCESS

RUNTIME BEHAVIOR

One request can become a governed, reusable technical artifact

Execution freezes the selected capability, team and agent versions; approval and reuse retain provenance.



Workflow Builder

Persisted node graph and run state



Human approval is resumable

Workflow approval uses the same review queue. A valid decision resumes the original run at the next node; repeated decisions are rejected.

Result envelope is schema-driven

Metrics, tables, charts, evidence, quality checks and boundaries render consistently across Assistant, Workforce and Verification.

Unsupported integrations fail honestly

Publish / Send only executes an explicitly supported integration. Unconfigured external send paths are blocked and recorded as unavailable.

IMPLEMENTATION SNAPSHOT

Technology stack, data contracts and current demo boundaries

This slide replaces the business-comparison content with what the repository actually runs and validates.

RUNTIME

Application stack

- Python 3.11+ / Flask service
- Responsive vanilla JavaScript frontend
- SQLite persistence
- Node.js frontend contract tests
- PDF generation and validation

PROVIDERS

AI and knowledge

- Ollama generation + embeddings
- Microsoft Copilot Studio / Direct Line
- Role-filtered document retrieval
- PDF, Office, CSV, text, Markdown, HTML
- No OCR for image-only PDFs

EXECUTORS

Registered analytics

- ADV / investor participation analysis
- Security liquidity and turnover analysis
- market_trades_demo.csv
- investor_participation_demo.csv
- daily_security_trading_demo.csv

Persistence map

SQLITE	missions, versions, worker events, agents, teams, capabilities, workflows, rooms, approvals, settings, RAG chunks
FILESYSTEM	controlled knowledge sources, synthetic CSV inputs, generated PDF reports
MEMORY ONLY	Local Analysis uploads; cleared at process restart or session deletion

Current boundaries

- Synthetic demonstration data and identities
- Local role checks—not production SSO
- External connectors require administrator configuration
- Provider failures surface as unavailable
- No production market feed is claimed

Primary implementation: server.py · index.html · static/js/workforce.js · bursaiq/*.py

TECHNICAL DEMO — NOT MARKET GUIDANCE